TornadoVM

beestream.ai



Java Goes CUDA-Native

Running LLMs and CUDA kernels with TornadoVM

Thanos Stratikopoulos
Beestream Ltd
University of Manchester

jitllm · pure Java on the GPU

```
$ ./jitllm run --gpu -v \
    -m Qwen3.8-27B-Q4_0.gguf

Device      : CUDA / NVIDIA GPU
Kernels     : Java → CUDA, TornadoVM
Native code : none
KV cache    : FP16, paged
CUDA graphs : on

> Why run LLMs in Java?
Because your stack already
does. ▌
```

0.94× of llama.cpp decode on Qwen3.8-27B, from Java

jtLLM

Kraków, Poland
jdd.org.pl

tornadovm.org

About me

TornadoVM maintainer, working on Java for GPUs.



Thanos Stratikopoulos

Principal Software Engineer · Beestream Ltd
Research Fellow · University of Manchester
TornadoVM Maintainer

@stratika
@thanos_str
@stratika.bsky.social
thanos.stratikopoulos@beestream.ai

Hands-on-Lab Repository

Scan for Lab X

Java runs the enterprise. NVIDIA runs compute.

Java teams own the services and the data. The fastest AI and HPC kernels ship as NVIDIA libraries.

WHERE JAVA ALREADY IS

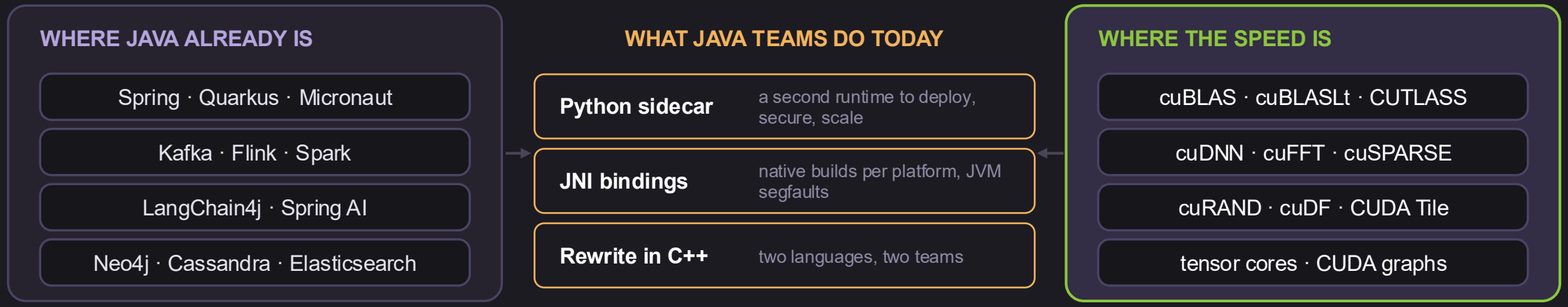
- Spring · Quarkus · Micronaut
- Kafka · Flink · Spark
- LangChain4j · Spring AI
- Neo4j · Cassandra · Elasticsearch

WHERE THE SPEED IS

- cuBLAS · cuBLASLt · CUTLASS
- cuDNN · cuFFT · cuSPARSE
- cuRAND · cuDF · CUDA Tile
- tensor cores · CUDA graphs

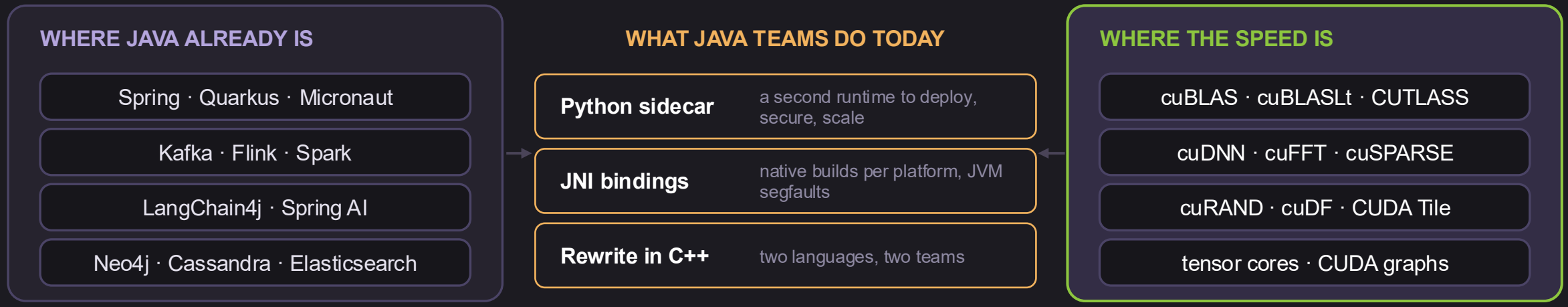
Java runs the enterprise. NVIDIA runs compute.

Java teams own the services and the data. The fastest AI and HPC kernels ship as NVIDIA libraries.



Java runs the enterprise. NVIDIA runs compute.

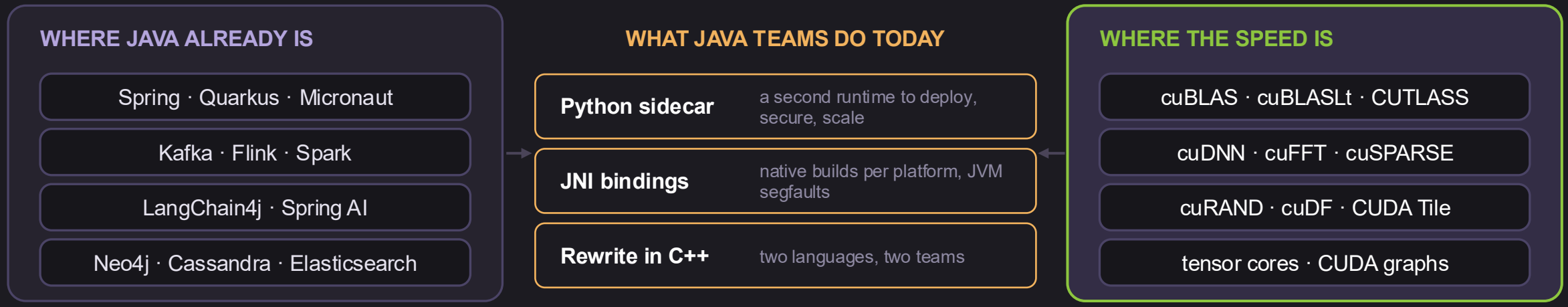
Java teams own the services and the data. The fastest AI and HPC kernels ship as NVIDIA libraries.



TornadoVM Hybrid API NVIDIA libraries (and Apple MLX) as a Maven dependency, on the JDK you already run.

Java runs the enterprise. NVIDIA runs compute.

Java teams own the services and the data. The fastest AI and HPC kernels ship as NVIDIA libraries.



TornadoVM Hybrid API NVIDIA libraries (and Apple MLX) as a Maven dependency, on the JDK you already run.

https://central.sonatype.com/namespace/io.github.beehive-lab

What we will cover

01	**TornadoVM, the concept**	what it is, and what it plugs into
02	**The API**	TaskGraph: what to run · TornadoExecutionPlan: how to run · what happens at execute()
03	**Going CUDA-native with the Hybrid API**	live in IntelliJ and on a Linux GPU server
04	**jitLLM**	a Java-native transformer at llama.cpp speed: API, serve, demo
05	**Summary**	what we saw today

TornadoVM

beestream.ai

01

TornadoVM
the concept

A Java framework that JIT-compiles selected methods to GPU kernels and manages the device around them.

The TornadoVM ecosystem

Any JDK from 21 to 27, three GPU backends, and the tools and integrations around them.

JDK 21–27 DISTRIBUTIONS

- OpenJDK
- Microsoft JDK
- GraalVM
- Azul Zulu
- Eclipse Temurin
- SapMachine
- Red Hat Mandrel

runs on any JDK

compiles per GPU

HARDWARE BACKENDS

OpenCL
AMD · Intel · NVIDIA

CUDA
NVIDIA

Metal
Apple Silicon

DEVELOPER TOOLS

- TornadoVMPulse
- TornadoViz
- TornadoInsight
- NVIDIA Nsight Systems

DOCKER IMAGES

NVIDIA and Intel images

DISTRIBUTION

Maven Central · SDKMAN!

INTEGRATIONS

- Spring Boot
- Quarkus
- LangChain4j
- Flink

Active since 2018 · 30+ contributors · 30+ research papers · used by the European Space Agency (ESA)

Quick install TornadoVM 7.3.1

Three ways to get started, with CUDA as the first-class NVIDIA backend.

1 Through SDKMAN!

Recommended

```
$ sdk list tornadovm
$ sdk install tornadovm 7.3.1-jdk22plus-cuda
```

2 Download the SDK directly

```
https://www.tornadovm.org/downloads
```

3 Clone & build from source

```
$ git clone https://github.com/beehive-lab/TornadoVM.git
$ cd TornadoVM && make BACKEND=cuda
$ source setvars.sh
```

Available versions

7.3.1-jdk22plus-cuda latest

7.3.1-jdk21-cuda

7.3.1-jdk22plus-opencl

7.3.1-jdk22plus-metal

Download TornadoVM SDK

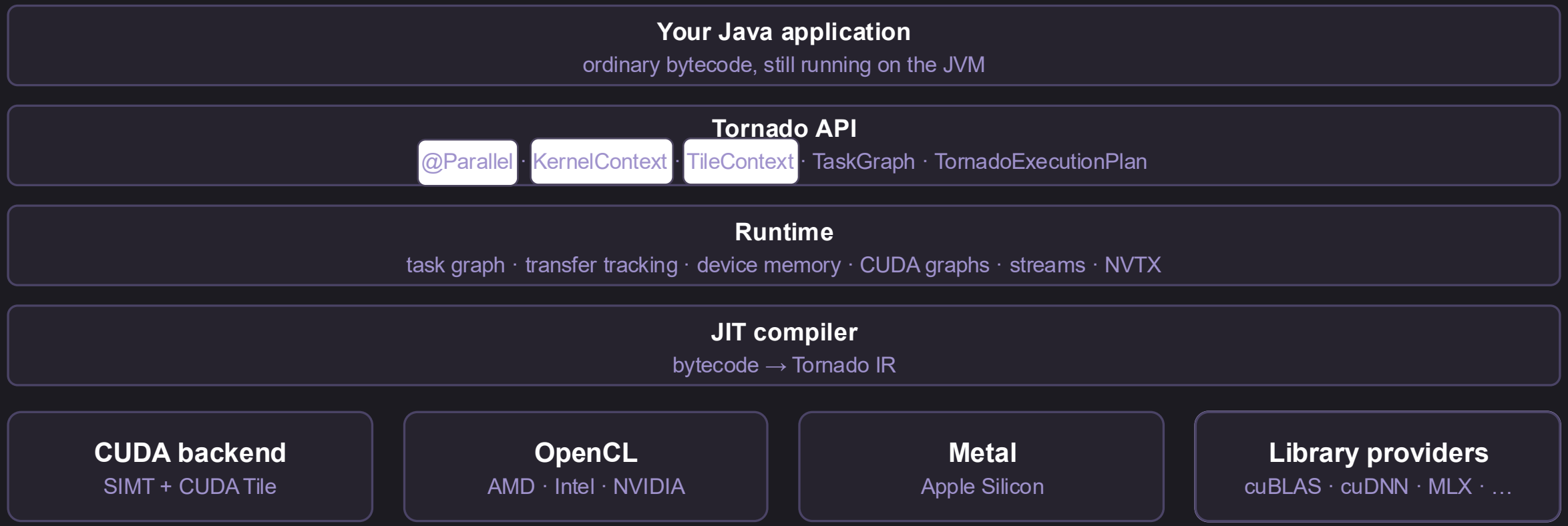
7.3.1 · JDK 22+ | CUDA | linux/x86_64

Download

```
// module-info.java
module my.app { requires tornado.api;
}
```

TornadoVM software stack

Runs on the JDK you already have. Not a JVM, not a binding layer.



02

The API
what to run, how to run

TaskGraph says what runs on the GPU and what data moves. TornadoExecutionPlan decides how it runs.

What to run, and how to run it

Two objects, two jobs. Every example in this talk uses both.

TaskGraph

defines what to run

- Tasks: Java methods, JIT-compiled for the GPU
- Data: what moves to the GPU, and when
- Order follows the data between tasks
- snapshot() freezes it as an immutable graph

TornadoExecutionPlan

defines how to run it

- Device, grid size, profiler, debug output
- CUDA graphs, streams, staged transfers
- Compiles once, then runs many times
- execute() hands back the results

describe the work in a **TaskGraph** → snapshot() → run it with a **TornadoExecutionPlan** → execute()

What to run, and how to run it

Two objects, two jobs. Every example in this talk uses both.

TaskGraph

defines what to run

- Tasks: Java methods, JIT-compiled for the GPU
- Data: what moves to the GPU, and when
- Order follows the data between tasks
- snapshot() freezes it as an immutable graph

TornadoExecutionPlan

defines how to run it

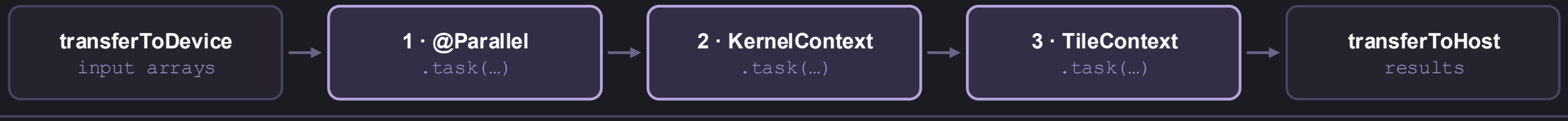
- Device, grid size, profiler, debug output
- CUDA graphs, streams, staged transfers
- Compiles once, then runs many times
- execute() hands back the results

describe the work in a **TaskGraph** → snapshot() → run it with a **TornadoExecutionPlan** → execute()

A TaskGraph is a chain of tasks

Each task is a Java method, JIT-compiled for the GPU. Chain them in one graph.

`new TaskGraph("pipeline")` chain tasks in order; data flows left to right



1 @Parallel

JIT · Java → CUDA C

A plain Java method. Annotate the loop; the JIT maps it to GPU threads.

```
for (@Parallel int i = 0; i < n; i++) {
  y.set(i, 2f * x.get(i));
}
```

2 KernelContext

JIT · Java → CUDA C

The CUDA thread model in Java: thread ids, shared memory, barriers.

```
int i = ctx.globalIdx;
s[ctx.localIdx] = x.get(i);
ctx.localBarrier();
```

3 TileContext

JIT · Java → CUDA Tile

Describe one tile; the tile compiler picks threads and tensor cores.

```
int r = tc.bidX();
int c = tc.bidY();
acc = tc.mma(a.load(r, k), b.load(k, c), acc);
```

TaskGraph: data stays on the GPU between tasks · how it runs (device, streams, one CUDA graph): **TornadoExecutionPlan**

A GPU kernel is a Java method

javac compiles it.

TornadoVM JIT-compiles it to CUDA C at runtime, specialised on the argument values it already knows.

--printKernel shows the exact CUDA it generated.

```
static void vectorAdd(FloatArray a, FloatArray b, FloatArray c) {
    for (@Parallel int i = 0; i < c.getSize(); i++)
        c.set(i, a.get(i) + b.get(i));
}

var tg = new TaskGraph("s0")
    .transferToDevice(FIRST_EXECUTION, a, b)
    .task("add", App::vectorAdd, a, b, c)
    .transferToHost(EVERY_EXECUTION, c);

try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
    plan.execute();
}
```

The whole TaskGraph API on one slide

All of it is Java on the TaskGraph and the execution result: no cudaMalloc, no cudaMemcpy.

What runs

`.task(…)`	@Parallel loop: TornadoVM picks the thread mapping
`.task(…, ctx, …)`	KernelContext first: globalIdx, local memory, barriers
`.task(…, tc, …)`	TileContext first: CUDA Tile, the grid counts tile blocks

What moves, and when

Call	Copies	Use for
`.transferToDevice(FIRST_EXECUTION, a)`	on the first run	weights, constants
`.transferToDevice(EVERY_EXECUTION, a)`	before every run	inputs that change
`.transferToHost(EVERY_EXECUTION, a)`	after every run	results read every time
`.transferToHost(UNDER_DEMAND, a)` `result.transferToHost(a)`	when you call it	iterative state
`result.transferToHost(new DataRange(a)` `.withOffset(o).withSize(s))`	only the slice	large outputs

Tasks decide what runs. Transfer modes decide what moves and when. Both live in the same few lines of host code.

Java kernels vs hand-written CUDA

Kernel time only (nsys) vs hand-written CUDA, same launch geometry. RTX 4090, TornadoVM 7.3.1.

98%

of CUDA C++ speed

register-tiled FP32 GEMM (1.02×)

≤3%

memory-bound gap

identical memory-sector counts

1.15×

faster, compute-bound

JIT specialises on argument values

TornadoVM

beestream.ai

03

The Hybrid API
your kernels + NVIDIA/Apple libraries

Java kernels, CUDA Tile kernels and vendor-optimized libraries in one task graph, on shared GPU buffers.

No more hassle: one Java shape for all

The same libraries, now a .libraryTask away, on your Java kernels' device buffers.

cuBLAS

Dense linear algebra

Use it for ML inference, embeddings, risk models

Saves you a tuned GEMM per GPU generation

`Sgemv · Sgemm · GemmEx`

cuBLASLt

Fused matmul

Use it for transformer layers: matmul + bias + GELU

Saves you writing fused epilogue kernels

`FP32 / FP16 / FP8`

cuFFT

Fourier transforms

Use it for signal and audio, spectra, time series

Saves you tuning FFTs for every size

`C2C · R2C · C2R · 2D`

cuDNN

Deep-learning primitives

Use it for CNNs, activations, LLM attention

Saves you conv and attention kernels per GPU

`conv2d · softmax · SDPA`

cuSPARSE

Sparse matrices

Use it for graphs (PageRank = SpMV), recommenders

Saves you balancing irregular sparse rows

`CSR SpMV · SpMM`

CUTLASS

Fused GEMMs

Use it for GEMM + bias + activation, native speed

Saves you C++ template metaprogramming

`S/H/B-GEMM · fused epilogues`

cuDF

GPU DataFrames

Use it for ETL, joins, group-by, CSV / Parquet reads

Saves you columnar kernels for relational ops

`readers · join · group-by`

MLX

Apple silicon · Metal

Use it for ML inference on the Mac: matmul, attention

Saves you hand-written Metal kernels per chip

`matmul · conv · fft · SDPA`

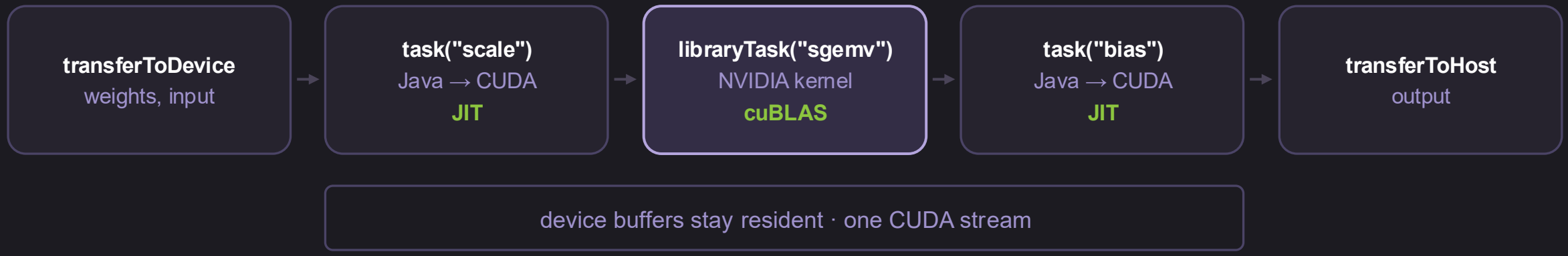
THE ANSWER · ONE CALL SHAPE

```
.libraryTask(
  name,
  Provider::method,
  args…)
```

Same device buffers as your Java kernels. No JNI. Also on Maven Central: cuRAND.

One task graph, two kinds of node

JIT-compiled Java kernels and NVIDIA library calls chained on the same builder.



Same buffer model

FloatArray, HalfFloatArray, BFloat16Array are off-heap segments the library reads directly.

Same consistency tracking

Access flags on library arguments drive transfers exactly like JIT task arguments.

Same CUDA graph

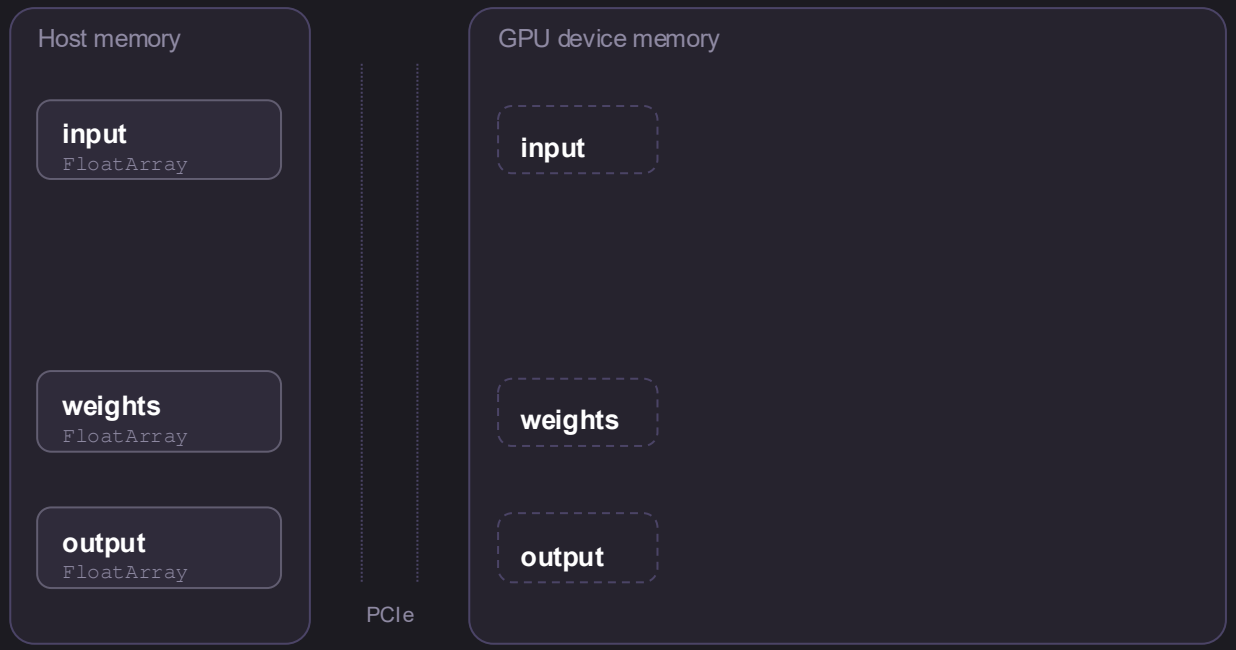
Library nodes are capturable: withCUDAGraph() replays the whole chain as one launch.

Java kernels and cuBLAS in one graph

FancyHybrid, the class in the demo: scale → sgemv → bias, captured as one CUDA graph.

```
var tg = new TaskGraph("hybrid")
  .transferToDevice(FIRST_EXECUTION, weights, bias)
  .transferToDevice(EVERY_EXECUTION, input)
  .task("scale", DenseLayer::scale, input, scaled)
  .libraryTask("sgemv", CuBlas::cublasSgemv,
      CUBLAS_OP_T.operation(), k, m, 1f,
      weights, k, scaled, 1, 0f, output, 1)
  .task("bias", DenseLayer::biasRelu, output, bias)
  .transferToHost(EVERY_EXECUTION, output);

try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
  plan.withGridScheduler(grid).withCUDAGraph().execute();
}
```

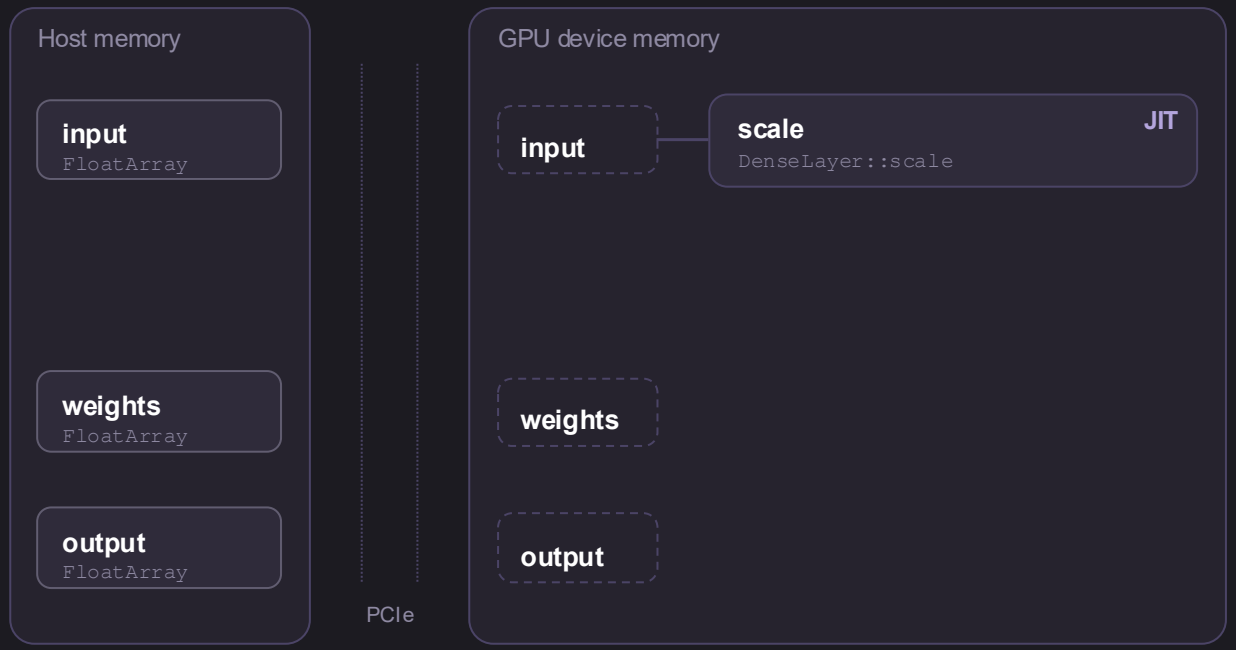


A TaskGraph starts by naming the host arrays it needs on the GPU: weights and bias once, input on every run. Their device buffers are allocated on the first execute().

Java kernels and cuBLAS in one graph

FancyHybrid, the class in the demo: scale → sgemv → bias, captured as one CUDA graph.

```
1  var tg = new TaskGraph("hybrid")
2    .transferToDevice(FIRST_EXECUTION, weights, bias)
3    .transferToDevice(EVERY_EXECUTION, input)
4    .task("scale", DenseLayer::scale, input, scaled)
5    .libraryTask("sgemv", CuBlas::cublasSgemv,
6        CUBLAS_OP_T.operation(), k, m, 1f,
7        weights, k, scaled, 1, 0f, output, 1)
8    .task("bias", DenseLayer::biasRelu, output, bias)
9    .transferToHost(EVERY_EXECUTION, output);
10
11 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
12   plan.withGridScheduler(grid).withCUDAGraph().execute();
13 }
```

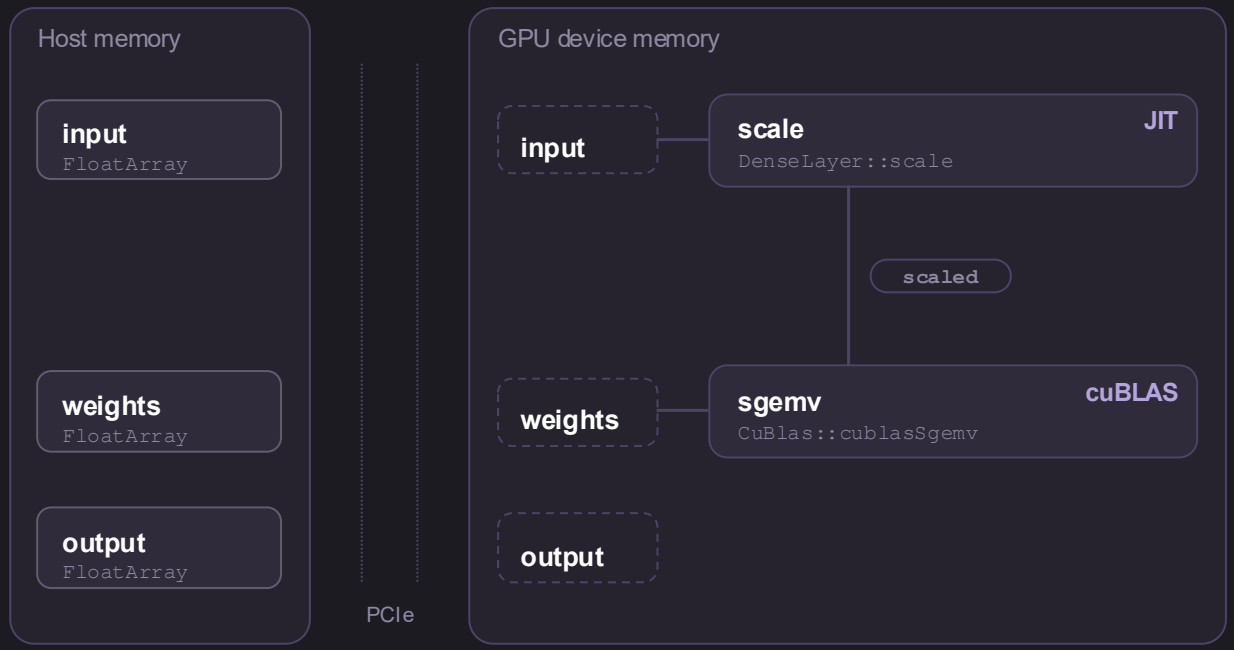


Task 1 is a Java kernel, JIT-compiled to CUDA C. Its output, scaled, is a device buffer the next task reads directly.

Java kernels and cuBLAS in one graph

FancyHybrid, the class in the demo: scale → sgemv → bias, captured as one CUDA graph.

```
1  var tg = new TaskGraph("hybrid")
2    .transferToDevice(FIRST_EXECUTION, weights, bias)
3    .transferToDevice(EVERY_EXECUTION, input)
4    .task("scale", DenseLayer::scale, input, scaled)
5    .libraryTask("sgemv", CuBlas::cublasSgemv,
6        CUBLAS_OP_T.operation(), k, m, 1f,
7        weights, k, scaled, 1, 0f, output, 1)
8    .task("bias", DenseLayer::biasRelu, output, bias)
9    .transferToHost(EVERY_EXECUTION, output);
10
11 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
12   plan.withGridScheduler(grid).withCUDAGraph().execute();
13 }
```

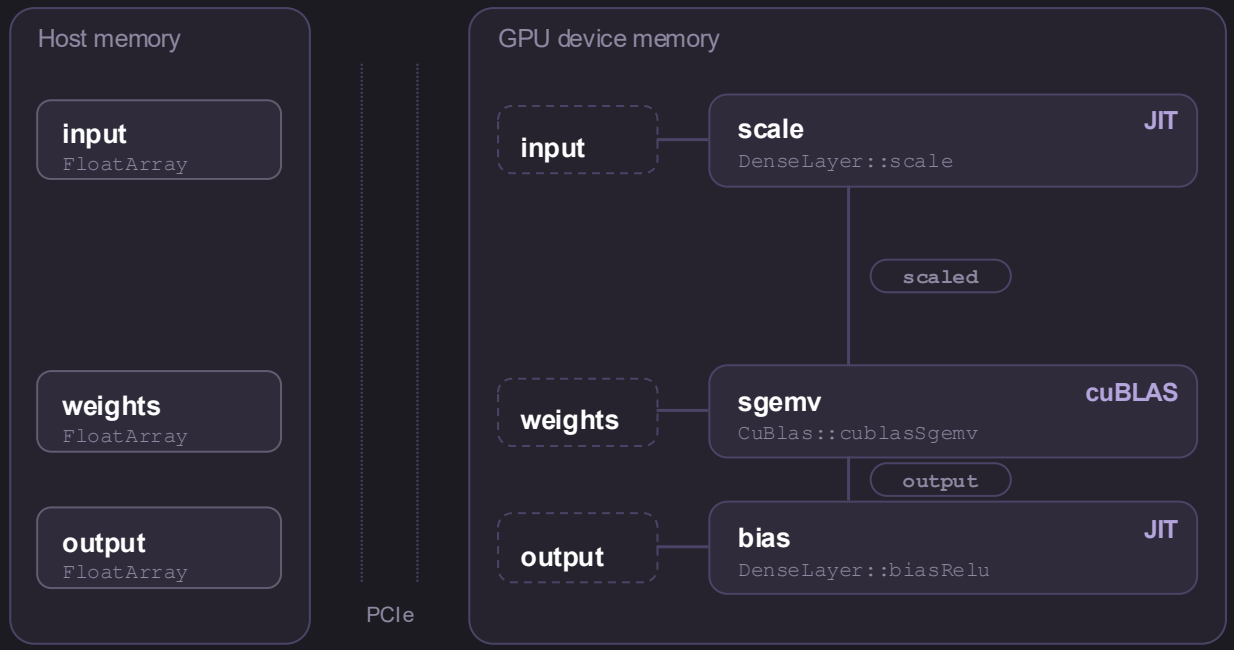


Task 2 is cuBLAS, added with libraryTask. It multiplies weights by scaled from the Java kernel: same buffer, no copy.

Java kernels and cuBLAS in one graph

FancyHybrid, the class in the demo: scale → sgemv → bias, captured as one CUDA graph.

```
1  var tg = new TaskGraph("hybrid")
2    .transferToDevice(FIRST_EXECUTION, weights, bias)
3    .transferToDevice(EVERY_EXECUTION, input)
4    .task("scale", DenseLayer::scale, input, scaled)
5    .libraryTask("sgemv", CuBlas::cublasSgemv,
6        CUBLAS_OP_T.operation(), k, m, 1f,
7        weights, k, scaled, 1, 0f, output, 1)
8    .task("bias", DenseLayer::biasRelu, output, bias)
9    .transferToHost(EVERY_EXECUTION, output);
10
11 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
12   plan.withGridScheduler(grid).withCUDAGraph().execute();
13 }
```

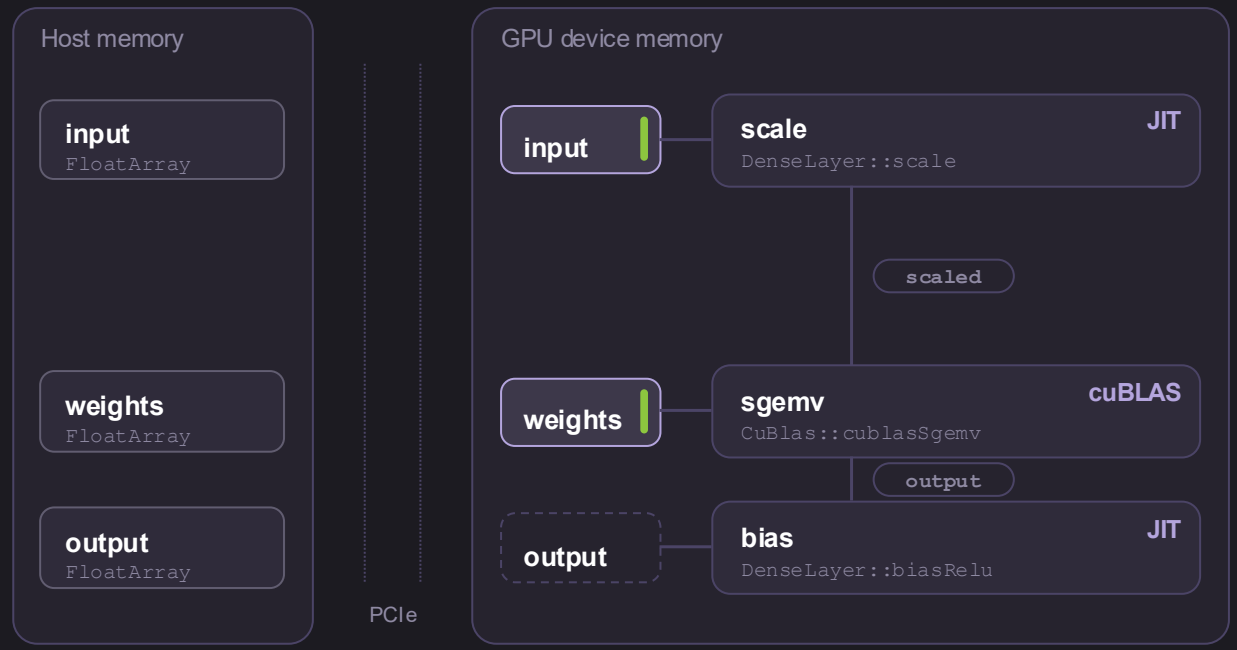


Task 3 is another Java kernel: it adds the bias and applies ReLU to output, in place, where cuBLAS wrote it.

Java kernels and cuBLAS in one graph

FancyHybrid, the class in the demo: scale → sgemv → bias, captured as one CUDA graph.

```
1  var tg = new TaskGraph("hybrid")
2    .transferToDevice(FIRST_EXECUTION, weights, bias)
3    .transferToDevice(EVERY_EXECUTION, input)
4    .task("scale", DenseLayer::scale, input, scaled)
5    .libraryTask("sgemv", CuBlas::cublasSgemv,
6        CUBLAS_OP_T.operation(), k, m, 1f,
7        weights, k, scaled, 1, 0f, output, 1)
8    .task("bias", DenseLayer::biasRelu, output, bias)
9    .transferToHost(EVERY_EXECUTION, output);
10
11 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
12   plan.withGridScheduler(grid).withCUDAGraph().execute();
13 }
```

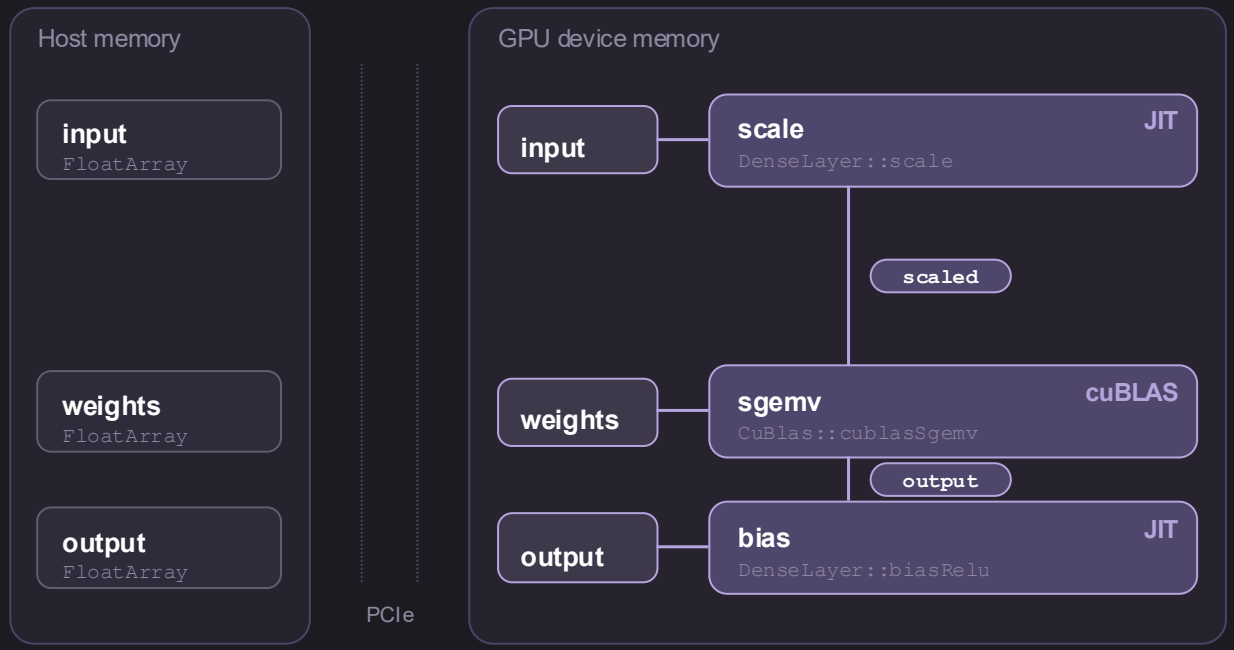


execute() starts by copying input into its device buffer, and weights and bias on the first run only.

Java kernels and cuBLAS in one graph

FancyHybrid, the class in the demo: scale → sgemv → bias, captured as one CUDA graph.

```
1  var tg = new TaskGraph("hybrid")
2    .transferToDevice(FIRST_EXECUTION, weights, bias)
3    .transferToDevice(EVERY_EXECUTION, input)
4    .task("scale", DenseLayer::scale, input, scaled)
5    .libraryTask("sgemv", CuBlas::cublasSgemv,
6        CUBLAS_OP_T.operation(), k, m, 1f,
7        weights, k, scaled, 1, 0f, output, 1)
8    .task("bias", DenseLayer::biasRelu, output, bias)
9    .transferToHost(EVERY_EXECUTION, output);
10
11 try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
12   plan.withGridScheduler(grid).withCUDAGraph().execute();
13 }
```



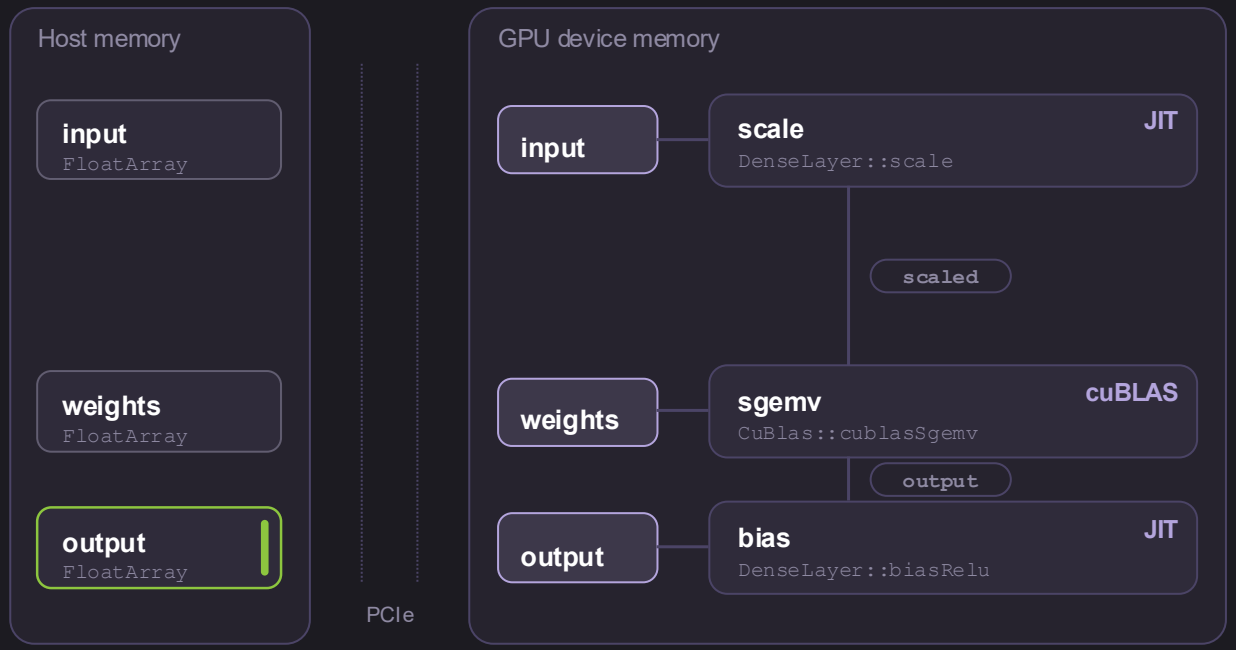
Then the three tasks run in order on one stream. withCUDAGraph() captures all of them, cuBLAS included, as one CUDA graph.

Java kernels and cuBLAS in one graph

FancyHybrid, the class in the demo: scale → sgemv → bias, captured as one CUDA graph.

```
var tg = new TaskGraph("hybrid")
  .transferToDevice(FIRST_EXECUTION, weights, bias)
  .transferToDevice(EVERY_EXECUTION, input)
  .task("scale", DenseLayer::scale, input, scaled)
  .libraryTask("sgemv", CuBlas::cublasSgemv,
      CUBLAS_OP_T.operation(), k, m, 1f,
      weights, k, scaled, 1, 0f, output, 1)
  .task("bias", DenseLayer::biasRelu, output, bias)
  .transferToHost(EVERY_EXECUTION, output);

try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
  plan.withGridScheduler(grid).withCUDAGraph().execute();
}
```



One CUDA graph: scale → sgemv → bias

Only output comes back. scaled never left the GPU, and weights and bias crossed PCIe once.

TornadoVM

beestream.ai

DEMO 1

In IntelliJ
the FancyHybrid class

The same ideas in a real class: the TaskGraph that says what to run, and the execution plan that says how.

Code · github.com/beehive-lab/tornadovm-hands-on-lab

```
demos/L15-maven-starter · FancyHybrid.java, FancyHybridMlx.java
./demos/L14-hybrid-mlx/run.sh
```

Hands-on-Lab Repository

Scan for Lab 14/15

TornadoVM

beestream.ai

DEMO 2

Linux server running FancyHybrid

The same class, running with the CUDA backend:
Java kernels and NVIDIA libraries in one task graph.

Code · github.com/beehive-lab/tornadovm-hands-on-lab

`./demos/L05-hybrid-cublas/run.sh · cuBLAS on a CUDA SDK, TornadoVM 7.3.1`

Hands-on-Lab Repository

Scan for Lab 05

tornadovm.org

CUDA runtime features also available from Java

Memory belongs to the task graph. CUDA's real mechanisms are typed, composable options on the plan.

```
try (var plan = new TornadoExecutionPlan(tg.snapshot())) {
    plan.withCUDAGraph()            // capture once, replay
        .withIntraPlanConcurrency() // DAG → stream pool
        .withStagedTransfers()      // pinned H2D ring
        .withGridScheduler(grid)    // explicit launch config
        .execute();
}

// memory is declared on the graph, not allocated by hand
.transferToDevice(FIRST_EXECUTION, weights)
.persistOnDevice(kvCache)
.transferToHost(EVERY_EXECUTION, logits);
```

What the plan options buy you

RTX 4090, TornadoVM 7.3.1, steady state. These workloads on this machine, not a general claim.

8.8×

CUDA-graph replay

≈300 → 35 µs per run (8.5–8.8×)

10.5×

JIT + cuBLAS graph

6 chains replayed, 9.9–10.5×

1.43×

stream pool vs one stream

1.29–1.43× measure your own

TornadoVM

beestream.ai

04

Running LLMs
a Java GPU inference engine

Everything we just saw powers a Java-native transformer that runs locally, as fast as llama.cpp.

Java developers in the LLM era

Java's legacy in the AI era calls for Java-native inference

REMOTE MODELS

Java application
Quarkus · Spring

→

LangChain4j
Orchestration · spans remote and local

→

Cloud LLM APIs
OpenAI · Anthropic · Gemini · Mistral

LOCAL MODELS · ON PREMISE privacy · security · custom weights · offline · domain-specific

HTTP to Python

vLLM · Ollama

a second runtime to deploy, secure and scale

JNI to C++

java-llama.cpp

native builds per platform, crashes in the JVM

Pure Java, CPU

llama3.java · Jlama · Quoxotic

portable, but CPU-bound

Pure Java, GPU

jitllm THIS TALK

one jar, four GPU vendors, in-process

Timeline of Java + Local Inference on GPUs

llama2.tornadovm · 2023 → llama3.java · CPU, Vector API → GPULlama3.java → **jitllm · 2026**

“

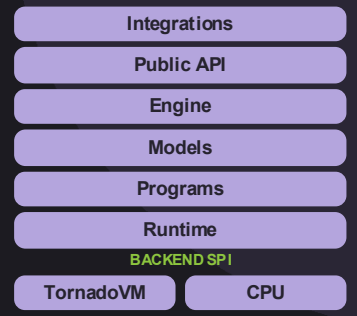
jitLLM:

Java Inference Tornado toolkit for LLMs that goes beyond Llama3

github.com/beehive-lab/jitllm

jitLLM

TornadoVM



jitLLM, layer by layer

TornadoVM is hidden at the bottom. On top ONLY modern Java.

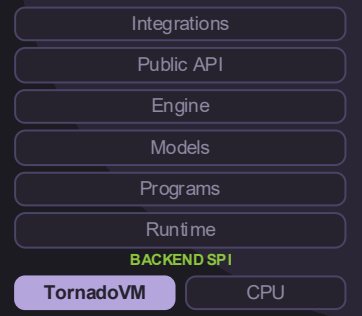
depends on

Integrations	CLI · OpenAI server · LangChain4j · Quarkus-LangChain4j · SpringAI
Public API	LocalModels · LocalModel · GenerationSession (13 stable types)
Engine	LLMEngine · Scheduler · KvCacheManager · BlockPool · PrefixCache
Models	GGUF → ModelProvider (ServiceLoader) → config + weights + tokenizer
Programs	backend-neutral ops: RMSNorm, RoPE, matvec, attention, SwiGLU, sampling
Runtime	DataType · tensor descriptors · memory planning · ExecutionPolicy

BACKEND SPI

TornadoVM backend
task graphs, Java kernels → CUDA · OpenCL · Metal

CPU backend
Vector API

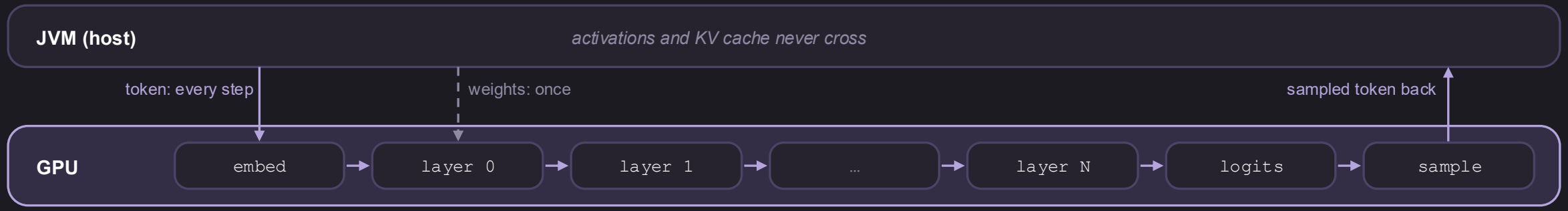


TornadoVM Backend: all the way down to JIT

It hands the TaskGraphs to TornadoVM, which JIT-compiles them into device code, once.



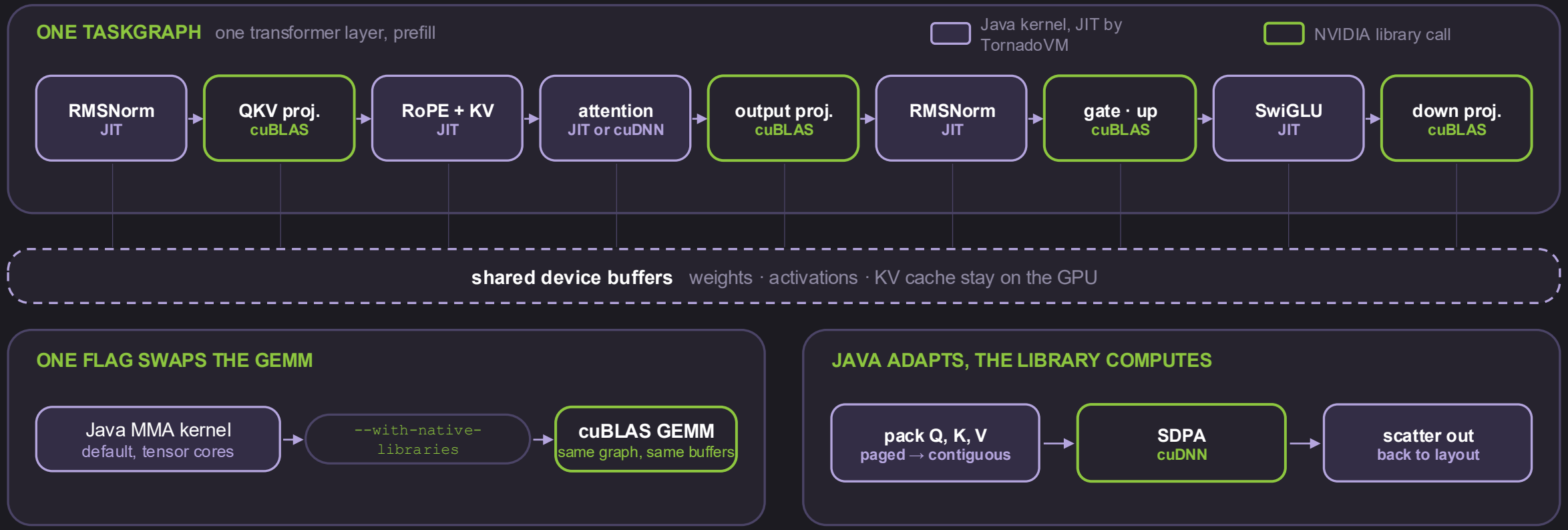
DATA STAYS ON THE GPU the JVM sends a token in and gets a token back



REPLAY after warm-up every token re-runs the same compiled kernels; with CUDA graphs, one launch replays them all. **195 tok/s**

JIT kernels mixed with NVIDIA libraries

JIT adapts on custom compute, the library does the heavy math.



WHERE WE ARE TODAY

Pure Java, at llama.cpp speed on GPUs

Same GPU, same GGUF weights. Every kernel written in Java and compiled by TornadoVM at runtime.

0.94× DECODE · 27B

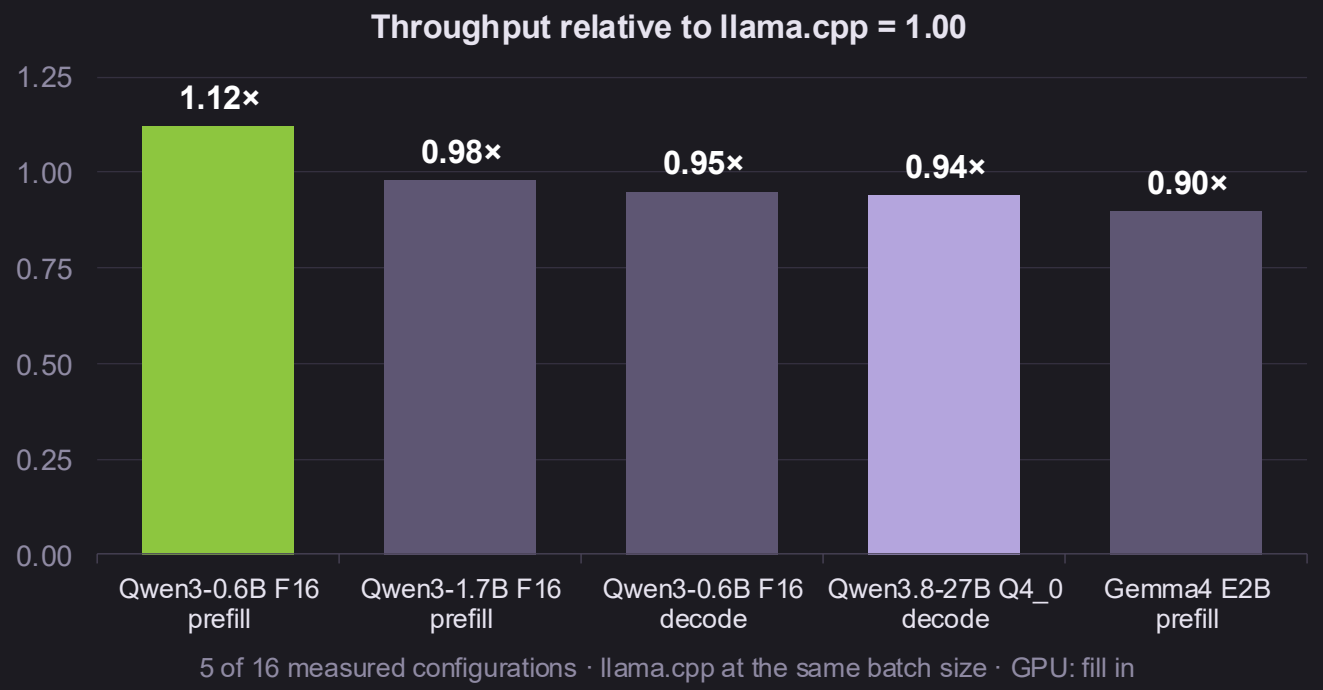
of llama.cpp decode on Qwen3.8-27B

40.5 vs 43.2 tok/s · Q4_0 · pure JIT, zero native code

1.12× PREFILL · HYBRID

faster than llama.cpp on prefill

Qwen3-0.6B F16 pp512 · 50,300 vs 44,805 tok/s · cuBLAS + Java



5 of 16 measured configurations · llama.cpp at the same batch size · GPU: fill in

No C++, no CUDA C, no JNI.

TornadoVM

beestream.ai

DEMO 3

jitLLM
on a local GPU

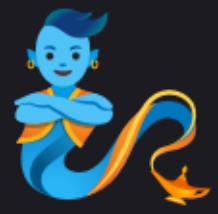
jitllm chat on a local GPU, then jitllm serve if time allows.

Code · github.com/beehive-lab/jitllm

`hands-on lab: demos/L06-jitllm/jitllm.sh chat · jitllm.sh serve`

Hands-on-Lab Repository

Scan for Lab 06 / Lab 10



tornadovm.org

TornadoVM

beestream.ai

DEMO 4

Quarkus, LangChain4j and jitLLM



jitLLM

Hands-on-Lab Repository

Scan for Lab 16

Takeaways

What we saw today.

01	**TornadoVM: GPUs from the JDK you run**	Java methods JIT-compiled to CUDA, OpenCL or Metal, with no JNI
02	**TaskGraph says what to run**	tasks, library calls and data transfers, in a few lines of Java
03	**TornadoExecutionPlan says how**	device, grid, CUDA graphs and streams; compile once, run many times
04	**The Hybrid API makes Java CUDA-native**	cuBLAS, cuFFT, cuDNN and your Java kernels in one CUDA graph
05	**jitLLM: LLMs in pure Java**	a transformer built from these pieces, at llama.cpp speed

Thank you! Questions?

Resources to take home.

1. **jitllm**
   `github.com/beehive-lab/jitllm`
2. **TornadoVM**
   `github.com/beehive-lab/TornadoVM`
3. **Maven Central**
   `io.github.beehive-lab:jitllm`
4. **LangChain4j provider**
   `docs.langchain4j.dev · langchain4j-jitllm`
5. **Hands-on lab: every demo in this talk**
   `github.com/beehive-lab/tornadovm-hands-on-lab`
6. **SDKMAN!**
   `sdkman.io/sdks/tornadovm`

Scan for jitllm

`github.com/beehive-lab/jitllm`

GET IN TOUCH

Thanos Stratikopoulos

@stratika

@thanos_str

@stratika.bsky.social

thanos.stratikopoulos@beestream.ai

Funded in part by the EU projects AERO, P2CODE, ENCRYPT and TANGO.

TornadoVM

beestream.ai

We value your feedback!

Rate this talk



